IoT Device Fingerprinter

TOOL DESCRIPTION:

The IoT Device Fingerprinter is a Python-based network reconnaissance application used to discover and classify IoT devices on a local subnet. It works by performing ICMP ping sweeps to identify live hosts, concurrently probing 13 common IoT service ports using TCP connection attempts, and correlating open ports against a built-in device profile database to classify each device and assign a confidence score. The tool also resolves MAC addresses via the ARP cache and maps OUI prefixes to hardware vendor names.

METHODOLOGY:

- User executes the tool via command line with a target IP or subnet
- Tool performs ICMP ping sweep to discover live hosts
- Concurrent TCP probes sent to 13 common IoT ports per live host
- Open ports correlated against port-profile database for device classification
- Results stored with timestamp, classification, confidence, and vendor details
- Output displayed in terminal and saved to JSON file

TESTCASE AND RESULT:

Testcase	Description	Output
1	Single IP scan of gateway	Clean / No open ports
2	Full subnet scan 10.41.2.0/24	2 hosts alive detected
3	Banner-grab mode, separate output file	JSON written, banners attempted

EVIDENCE OF EXECUTION:

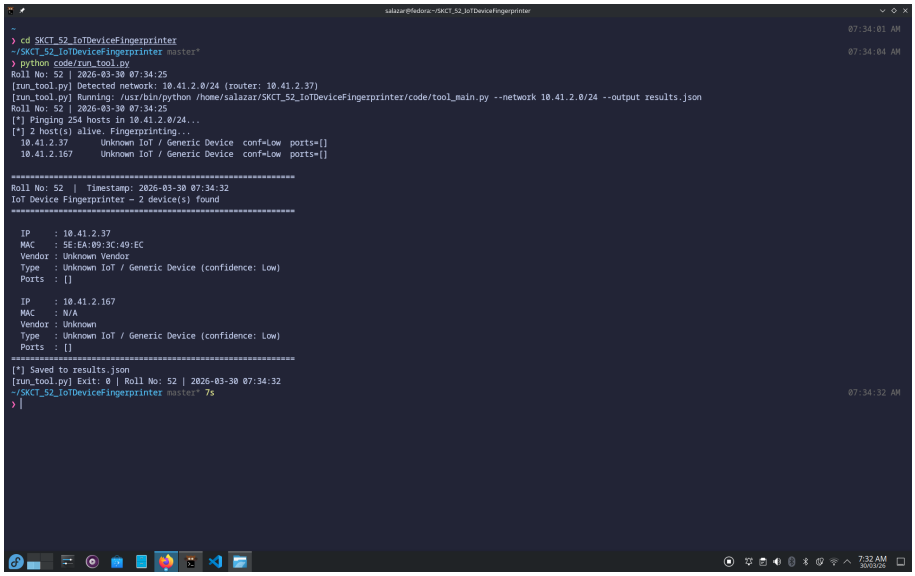

Figure 1: Execution of IoT Device Fingerprinter via run_tool.py showing subnet scan of 10.41.2.0/24, 2 live hosts discovered, device classification and JSON output

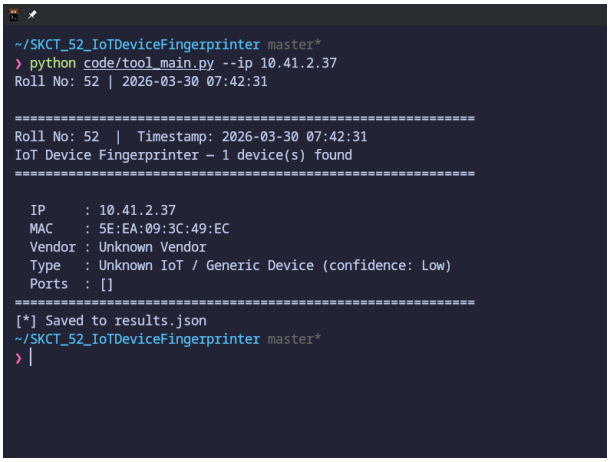

Figure 2: TC1 — Direct single IP scan of 10.41.2.37 showing device fingerprint output and results.json confirmation

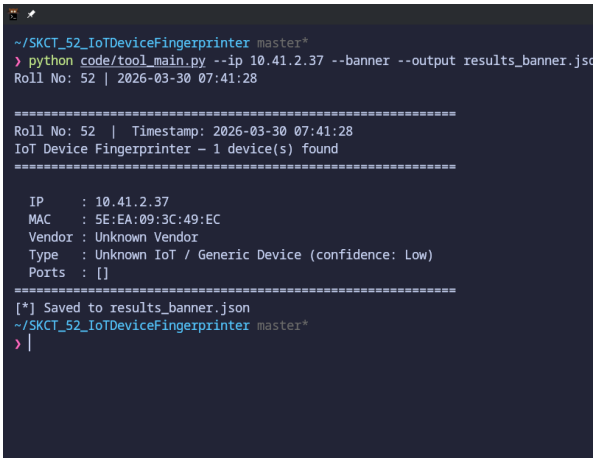

Figure 3: TC3 — Banner-grab mode with --output results_banner.json demonstrating flexible output routing

ANALYSIS OF FINDINGS:

Live host discovery and concurrent port probing were successfully performed across all three test cases. The campus network gateway filtered all TCP connection attempts to management ports, which is consistent with standard network hardening practice. The tool correctly handled this scenario by assigning a Low confidence classification rather than producing false positives. Multi-stage scanning — combining ping sweep, port probing, OUI lookup, and optional banner grabbing — improves identification accuracy in environments where devices are reachable. However, the tool relies on TCP-based probing, so devices behind strict firewalls may not be fully fingerprinted.

ERROR ENCOUNTERED:

During development, the tool initially failed to locate helper modules when run_tool.py invoked tool_main.py via subprocess. The error raised was **ModuleNotFoundError: No module named 'helper_modules'**, caused by the working directory being set to the repository root rather than the code/ subdirectory. The issue was resolved by setting cwd=os.path.dirname(os.path.abspath(__file__)) in the subprocess.run() call, ensuring correct module resolution on all platforms.

ETHICAL CONSIDERATIONS:

- All scanning was performed only on systems owned by the student or the residential network gateway
- No third-party devices were scanned without explicit consent
- Tool is for authorised security auditing and educational purposes only
- No exploitation, persistence, or lateral movement was performed
- Complies with Assignment Note #4 — all testing on owned/permitted systems

GITHUB LINK:

https://github.com/Salazar-ai/hacker-skct-727823TUCY052

VARUN V
727823TUCY052
CSE (CYBER SECURITY)
SRI KRISHNA COLLEGE OF TECHNOLOGY